

Project overview : PGM image processing

❖ **Presented by :**

**Manar BOUAMAR &
Hidaya LIO-EDDINE.**

❖ **Supervised by :**

Mr. Khalid BOUKHDIR.

❖ **Academic Year: 2025–2026.**

❖ **Class : GLD-1.**

- **Project Goals:**

The main goal of this project is to build a C program that can handle PGM (Portable Gray Map) images. This project helps in understanding how grayscale images work by writing code to manage memory, manipulate pixels directly, and perform geometric transformations. It also explores a theoretical way to hide text data inside an image using steganography.

- **Core Data Structure:**

The image is stored using a custom struct called **Image**. This structure holds all the necessary metadata and the actual image data:

- **Rows:** The height of the image (number of pixel rows).
- **Cols:** The width of the image (number of pixel columns).
- **Max_gray:** The highest possible value for a pixel (usually 15 or 255).
- **Pixels:** A 2D integer array, allocated dynamically, that contains the gray level for every coordinate.

- **Project File Structure:**

The entire project is contained within a single source file, organized into the following functional sections to ensure clarity and maintainability:

- **Header & Data Structures:** At the top of the file, we include necessary libraries (like `stdio.h`, `stdlib.h`, and `math.h`) and define the `Image` structure used throughout the program.
- **Memory Management:** This section contains `create_image` for dynamic memory allocation and `free_image` to ensure all allocated resources are properly cleaned up.
- **Core Utilities & Pixel Access:** Includes helper functions to retrieve image dimensions and safely get or set pixel values, preventing out-of-bounds errors.
- **Image Transformations:**

-Negation: Logic to invert pixel values.

-Rotation: Implementation of general rotation and the specific rotate_theta function, which uses trigonometric formulas for precise coordinate mapping.

- **Steganography (Encoding/Decoding):** A dedicated section for the strategy used to hide and extract secret messages within the image data.
- **Main Entry Point:** The main () function at the end of the file serves as the test bench, where the workflow of loading, processing (rotating/negating), and saving the image is executed.
- **Steganography (Data Hiding Strategy):**

The project implements a strategy to hide secret messages without visibly altering the image:

- **LSB (Least Significant Bit) Method:** Each character (8 bits) is split into four 2-bit pieces.
- **Encoding:** These pieces are injected into the 2 least significant bits of four different pixels.
- **Decoding:** The message is reconstructed by extracting these bits from the pixels until a "Null" character (\0) is reached.

- **Conclusion**

Building this project provides a solid introduction to low-level programming in C. It covers the basics of file structures, dynamic memory allocation, and how to apply mathematical concepts to digital media.